

Workbench Roadmap

Updated after completion of the v0.1 feature set. Workbench is a React + TypeScript learning project that can later evolve into a server-backed, cloud-hosted application and potentially become an early module within Atlas.

Guiding principles

- **Learn by building:** each milestone should introduce a small number of new concepts through working software.
- **Keep slices small:** avoid premature architecture and only add complexity when the current version gives us a reason to.
- **Test behaviour:** continue using Red - Green - Refactor for user-facing behaviour where it adds learning value.
- **Build toward usefulness:** each version should leave Workbench a little more capable as an actual task tool, not just a collection of exercises.

v0.1.0 - First usable task loop

Status: Complete

- React + TypeScript + Vite baseline
- Task list rendering
- Add tasks
- Toggle tasks complete / incomplete
- Delete tasks
- Accessible labels and task actions
- Vitest + React Testing Library behavioural tests
- Basic dark-theme styling
- UUID-based task IDs
- README and release cleanup

v0.2 - Local usability

Status: Next

- Edit existing tasks
- Filter tasks (for example: all / open / completed)
- Persist tasks in localStorage
- Refine component structure where the growing UI now justifies it
- Expand tests around the new behaviour

v0.3 - Shared client state

Status: Planned

- Introduce Zustand when state begins to span more than one local component boundary
- Move appropriate client-side state out of App
- Keep server-state concerns separate from client UI state
- Preserve current behaviour through tests during the refactor

v0.4 - Server API

Status: Planned

- Introduce a small REST API
- Use Node / Express or a similarly lightweight server
- Validate API inputs with Zod
- Add persistent database storage
- Replace local-only persistence with an API-backed task model

v0.5 - Server-state integration

Status: Planned

- Introduce TanStack Query
- Fetch task data through the API
- Use mutations for create / edit / toggle / delete
- Use cache invalidation and refetching rather than duplicating server data in client state

v0.6 - Identity and cloud data

Status: Planned

- Add authentication
- Associate data with individual users
- Move persistence to a cloud-hosted database
- Review security boundaries and validation across client / API / database

v1.0 - Deployable portfolio milestone

Status: Planned

- Deploy the client and API
- Responsive layout
- Accessibility review
- Useful automated test coverage
- Polished README with screenshots and architecture notes
- Stable, demonstrable end-to-end workflow

How the roadmap evolves

The roadmap is intentionally directional rather than rigid. During v0.1, task creation and deletion were pulled forward because they were a natural next step in the React and TDD learning flow. Rather than treating that as scope drift, this revision records the software as it actually exists and moves the remaining work forward.

The same rule should continue to apply: if a feature is the clearest way to learn the next concept and can be delivered in a small, coherent slice, it can move earlier. The roadmap should then be updated to reflect reality.

Near-term focus: v0.2

- **Edit:** complete the basic CRUD-style task interaction by allowing a task title to be changed.
- **Filter:** introduce derived views of the same task collection without yet requiring a global state library.
- **Persist locally:** make Workbench survive a browser refresh before introducing a server.
- **Refactor only when earned:** if App or the task components become awkward as these features arrive, use that pressure to decide what should be extracted.

Longer-term direction

Workbench should remain a deliberately small learning surface while it progresses through local state, shared client state, API integration, persistence, server-state management, authentication and deployment. The goal is to experience the architectural transitions rather than jump directly to the final stack.

If it continues to prove useful, Workbench may later become a concrete module or proving ground for Atlas. That possibility should influence compatibility and learning choices, but it should not force Atlas-scale architecture into Workbench prematurely.

Version	Primary learning emphasis
v0.1	React components, props, local state, events, accessibility, behavioural testing
v0.2	Editing, filtering, local persistence, modest component refinement
v0.3	Shared client state with Zustand
v0.4	REST APIs, validation and persistent backend storage
v0.5	Server-state management with TanStack Query
v0.6	Authentication, users and cloud data
v1.0	Deployment, accessibility, documentation and portfolio readiness